perplexity

show me the users in the db

You can list all users with a simple `SELECT` on your users table once you're connected to the DB.

Generic SQL (most ORMs/backends)

```
SELECT * FROM users;
```

If your table has a different name (e.g. `app_user`, `auth_user`, `accounts`), replace `users` accordingly. [1] [2]

If you meant DB server users (not app users)

- PostgreSQL (SQL way):

  ```
  SELECT rolname AS username FROM pg_roles;
  ```[^1_3][^1_4]

- MySQL/MariaDB server users:

  ```
  SELECT User, Host FROM mysql.user;
  ```[^1_5][^1_6]

Tell me which DB/stack you're using (e.g. "Postgres via SQLAlchemy", "Firebase", "Supabase") and I can give you the exact command or code snippet.

⁂

from the vs code terminal

From your VS Code terminal in that project, do:

1. Activate venv (if not already):

```
cd /workspaces/e-farmersmarket
source venv/bin/activate  # or just use the full python path below
```

2. Open a Python shell and query via SQLAlchemy:

```
/workspaces/e-farmersmarket/venv/bin/python
```

Then inside the Python REPL:

```
from app import app, db
from models import User

with app.app_context():
    for u in User.query.all():
        print(u.id, u.email, u.username, u.role)
```

Press Ctrl+D (or `exit()`) to leave the shell when done.[7] [8]

⁂

bash: from: command not found

bash: from: command not found
bash: syntax error near unexpected token `(' bash: syntax error near unexpected token` ('
bash: syntax error near unexpected token `u.id,'\

Those errors mean you pasted Python code directly into the Bash shell. You need to first enter the Python REPL, then run the Python lines there.

Do this in the VS Code terminal:

1. Go to your project folder:

```
cd /workspaces/e-farmersmarket
```

2. Start Python (using your venv interpreter):

```
/workspaces/e-farmersmarket/venv/bin/python
```

You should now see a prompt like:

```
Python 3.12.2 (...
>>>
```

3. Now, at the `>>>` prompt, paste the Python code:

```
from app import app, db
from models import User

with app.app_context():
    for u in User.query.all():
        print(u.id, u.email, u.username, u.role)
```

4. When you are done, exit the Python shell:

```
exit()
```

or press Ctrl+D.[11] [12] [13]

⁂

im still able to login with producer role from customer login, add a new auth line. when the email is entered, check if the user existst in the db. we already do this, now if the user exists, password should only show if the existing user role is customer. otherwise print a message saying (please use Business Login)

You only need to change the **customer** `/login` view so it refuses producer emails after the first step.

Assuming your current login route roughly looks like this:

```
@app.route("/login", methods=["GET", "POST"])
def login():
    step = request.args.get("step", "email")
    email = request.args.get("email", "").strip().lower()

    if step == "email":
        form = EmailLoginForm()
        if form.validate_on_submit():
            email = form.email.data.strip().lower()
            user = User.query.filter_by(email=email).first()
            if user:
                # currently you always go to password
                return redirect(url_for("login", step="password", email=email))
            else:
                flash("No account with that email. Please sign up.", "warning")
                return redirect(url_for("signup", email=email))

        return render_template("login.html", step="email", email=email, form=form)

    elif step == "password":
        form = PasswordLoginForm()
        ...
```

Change the `step == "email"` block to check the role before redirecting to the password step:

```
@app.route("/login", methods=["GET", "POST"])
def login():
    step = request.args.get("step", "email")
    email = request.args.get("email", "").strip().lower()

    if step == "email":
        form = EmailLoginForm()
        if form.validate_on_submit():
            email = form.email.data.strip().lower()
            user = User.query.filter_by(email=email).first()

            if user:
                # NEW: block producer here
                if user.role != "CUSTOMER":
                    flash("Bitte nutzen Sie den Business Login.", "warning")
                    # stay on email step, do NOT show password
                    return redirect(url_for("login"))
                # only customers may see password step
                return redirect(url_for("login", step="password", email=email))
            else:
                flash("Kein Konto mit dieser E-Mail gefunden. Bitte registrieren.", "warn
                return redirect(url_for("signup", email=email))

        return render_template("login.html", step="email", email=email, form=form)

    elif step == "password":
        form = PasswordLoginForm()
        ...
```

Key points:

- Lookup user by email immediately after the email form submit.
- If `user.role != "CUSTOMER"`, flash `"Bitte nutzen Sie den Business Login."` and stay on the email step (no password form).
- Only redirect to the password step if `user.role == "CUSTOMER"`.

This way, a producer email can never reach the customer password screen; they are told to use Business Login instead.[21] [22] [23]

⁂

if step == "email":

```
    form = EmailOnlyLoginForm()
    if form.validate_on_submit():
        email = form.email.data.strip().lower() #get email from form
        user = User.query.filter_by(email=email).first() #check if user email exists
        if user and user.role != "CUSTOMER":
            form.email.errors.append("Dieser Login ist nur für Kunden. Bitte verwenden Si
            return redirect(url_for("login", step="password", email=email))
        else:
```

```
            return redirect(url_for("login", step="signup", email=email))
    return render_template("login.html", step="email", email=email, form=form)
```

Right now your logic always redirects away from the email step, even when the role is wrong. You want to **stay** on the email step and only go to password for customers.

Fix it like this:

```
if step == "email":
    form = EmailOnlyLoginForm()

    if form.validate_on_submit():
        email = form.email.data.strip().lower()  # get email from form
        user = User.query.filter_by(email=email).first()  # check if user email exists

        if user:
            if user.role != "CUSTOMER":
                # stay on email step and show error on the same form
                form.email.errors.append(
                    "Dieser Login ist nur für Kunden. Bitte verwenden Sie den Business-Lo
                )
                # IMPORTANT: do NOT redirect here
                return render_template("login.html", step="email", email=email, form=form

            # user exists and is a CUSTOMER → go to password step
            return redirect(url_for("login", step="password", email=email))

        # no user with this email → go to signup flow
        return redirect(url_for("login", step="signup", email=email))

    # initial GET or failed validation (required, email format, etc.)
    return render_template("login.html", step="email", email=email, form=form)
```

Why:

- For producer role (`user.role != "CUSTOMER"`), you render the same template so WTForms can display `form.email.errors` instead of redirecting, which would lose errors.[31] [32] [33]
- Only customers reach `step="password"`.
- Non-existing emails go to your signup step as before.

⁂

i can enter an email address that is not in the standart format "username@domina.com", this should be expected

To force a proper email format, add the **Email** validator to your login form field (and make sure `email_validator` is installed).

1. Form class with email validation

In your `forms.py` (or wherever `EmailOnlyLoginForm` lives):

```
from flask_wtf import FlaskForm
from wtforms import StringField, SubmitField
from wtforms.validators import DataRequired, Email

class EmailOnlyLoginForm(FlaskForm):
    email = StringField(
        "E-Mail",
        validators=[
            DataRequired(message="Bitte geben Sie Ihre E-Mail-Adresse ein."),
            Email(message="Bitte geben Sie eine gültige E-Mail-Adresse ein."),
        ],
    )
    submit = SubmitField("Weiter")
```

The `Email` validator checks that input looks like `username@domain.com` and adds an error if not.[41] [42] [43] [44]

2. Ensure the package is installed

In your venv:

```
pip install email_validator
```

WTForms uses this package under the hood for robust email checks.[45]

3. Template shows the error

In `login.html`, make sure you render field errors, e.g.:

```
<div class="form-group">
  {{ form.email.label }}
  {{ form.email(class="form-control") }}
  {% for err in form.email.errors %}
    <div class="text-danger small">{{ err }}</div>
  {% endfor %}
</div>
```

Now invalid formats will be rejected before your `User.query.filter_by(email=...)` logic runs, and the user will see a clear error message under the email field.[46] [47] [48]

⁂

use the same user role check for business login

For the **business** login you just invert the role condition: only `PRODUCER` emails may proceed to password, everyone else gets an error.

Assuming your `/business` route is similar to `/login`:

```
@app.route("/business", methods=["GET", "POST"])
def business():
    step = request.args.get("step", "email")
    email = request.args.get("email", "").strip().lower()

    if step == "email":
        form = EmailOnlyLoginForm()
        if form.validate_on_submit():
            email = form.email.data.strip().lower()
            user = User.query.filter_by(email=email).first()

            if user:
                # only allow PRODUCER here
                if user.role != "PRODUCER":
                    form.email.errors.append(
                        "Dieser Login ist nur für Produzenten. Bitte verwenden Sie den Ku
                    )
                    return render_template(
                        "business.html", step="email", email=email, form=form
                    )

                # correct role → go to password step
                return redirect(url_for("business", step="password", email=email))

            # no user with this email → go to business signup
            return redirect(url_for("business", step="signup", email=email))

        return render_template("business.html", step="email", email=email, form=form)

    elif step == "password":
        form = PasswordLoginForm()
        # your existing password check: verify password, login_user(user), etc.
        ...
```

This mirrors the customer login:

- Customer `/login`: only `role == "CUSTOMER"` can reach the password step; others get “Bitte verwenden Sie den Business-Login.”
- Business `/business`: only `role == "PRODUCER"` can reach the password step; others get “Bitte verwenden Sie den Kunden-Login.”

what measures should i take if user is logged in and clickes business login

If a logged-in user clicks “Business Login”, you should check their auth state and role, then either redirect or log them out before continuing.

1. Basic rule with `current_user`

In `business` route:

```
from flask_login import current_user, logout_user
from flask import redirect, url_for, flash, request
```

Add this at the top of the view:

```
@app.route("/business", methods=["GET", "POST"])
def business():
    step = request.args.get("step", "email")

    # If already logged in
    if current_user.is_authenticated:
        if current_user.role == "PRODUCER":
            # Already a producer → send to producer dashboard
            return redirect(url_for("producer_dashboard"))
        else:
            # Logged in as CUSTOMER (or other) but wants business login
            # Option A: forbid
            flash("Sie sind bereits als Kunde eingeloggt. Bitte zuerst abmelden, bevor Si
            return redirect(url_for("index"))
            # Option B (if you prefer): auto-logout, THEN show business login
            # logout_user()
            # flash("Sie wurden ausgeloggt. Bitte melden Sie sich jetzt als Produzent an.
            # return redirect(url_for("business"))
```

Then keep the email/password step logic as you already have it for producers.[51] [52] [53]

2. Mirror behavior on customer `/login`

Do the same idea for customer login:

```
@app.route("/login", methods=["GET", "POST"])
def login():
    step = request.args.get("step", "email")

    if current_user.is_authenticated:
        if current_user.role == "CUSTOMER":
            return redirect(url_for("customer_dashboard"))
        else:
```

```
            flash("Sie sind bereits als Produzent eingeloggt. Bitte zuerst ausloggen oder
            return redirect(url_for("index"))
```

This ensures:

- A logged-in producer never sees the business login form again; they go straight to their dashboard.
- A logged-in customer cannot silently “switch” to producer via Business Login without first logging out (or being auto-logged-out if you choose Option B).[54] [55] [53]

⁂

no meant logically, a customer is using the website, is logged in already so logged in user role is customer. what should the business page do, should it be just a marketing page?

Yes, for a logged-in **customer**, the Business page should behave like a **marketing/landing page**, not a second login.

Recommended behavior

- If `current_user.is_authenticated` and `current_user.role == "CUSTOMER"`:
  - Show a business **info page**: benefits of becoming a producer, how it works, pricing, FAQ.
  - Include a clear CTA like “Jetzt als Produzent registrieren” that:
    - Either tells them “Producer accounts are separate, please log out and sign up as business”, or
    - Starts a producer-onboarding flow that requires confirmation and possibly separate credentials.
- If `current_user.is_authenticated` and `current_user.role == "PRODUCER"`:
  - Skip the marketing and redirect straight to the **producer dashboard**.
- If not logged in:
  - Business page can show marketing plus the **email step** for business login/signup (what you already built).

This keeps mental model clean:

- Customer: Business page = information + call to action, no second login form.
- Producer: Business page = entry point to their dashboard.

lets do this if role is customer then use it as marketing page, if user role producer then he is already logged in and show the dashboard directly.

You can implement that behavior directly in the `/business` route with a simple role check.

Route logic

```
from flask_login import current_user, login_required
from flask import render_template, redirect, url_for

@app.route("/business", methods=["GET", "POST"])
def business():
    # 1) User is logged in as CUSTOMER → marketing page
    if current_user.is_authenticated and current_user.role == "CUSTOMER":
        return render_template("business_marketing.html")

    # 2) User is logged in as PRODUCER → go straight to dashboard
    if current_user.is_authenticated and current_user.role == "PRODUCER":
        return redirect(url_for("producer_dashboard"))

    # 3) Not logged in → show normal business login/signup flow
    step = request.args.get("step", "email")
    email = request.args.get("email", "").strip().lower()

    if step == "email":
        form = EmailOnlyLoginForm()
        if form.validate_on_submit():
            email = form.email.data.strip().lower()
            user = User.query.filter_by(email=email).first()

            if user:
                if user.role != "PRODUCER":
                    form.email.errors.append(
                        "Dieser Login ist nur für Produzenten. Bitte verwenden Sie den Ku
                    )
                    return render_template("business.html", step="email", email=email, fo

                return redirect(url_for("business", step="password", email=email))

            return redirect(url_for("business", step="signup", email=email))

        return render_template("business.html", step="email", email=email, form=form)

    elif step == "password":
        form = PasswordLoginForm()
        # verify password for PRODUCER and login_user(...)
        ...
```

Templates

- `business_marketing.html`: pure marketing content for logged-in customers (what is FoodLoop for producers, how to join, etc.).
- `business.html`: your existing multi-step login/signup UI for producers and non-logged-in visitors.

This matches your rule:

- Logged-in **customer** → sees **marketing page**.
- Logged-in **producer** → goes **directly to dashboard**.
- **Not logged in** → sees business login/signup flow.[61] [62] [63]

⁂

1. https://dev.mysql.com/doc/en/select.html
2. https://www.w3schools.com/mysql/mysql_select.asp
3. https://www.dbvis.com/thetable/postgres-list-users-two-different-approaches/
4. https://stackoverflow.com/questions/18866881/how-to-get-the-list-of-all-database-users
5. https://dataedo.com/kb/query/sql-server/list-users-in-database
6. https://www.sqlservercentral.com/forums/topic/list-of-all-users-for-all-databases
7. work.web_development
8. skills.programming
9. https://www.perplexity.ai/search/ba6864f2-7243-4660-aae5-b3e9fdf62aca
10. https://www.perplexity.ai/search/b3fcf93a-4e6e-4047-aeab-9f6dcd8f27ad
11. https://code.visualstudio.com/docs/python/run
12. https://stackoverflow.com/questions/61808776/how-do-i-open-the-interactive-shell-repl-in-visual-studio-code
13. https://code.visualstudio.com/docs/python/python-quick-start
14. https://stackoverflow.com/questions/9673730/interacting-with-bash-from-python
15. https://www.youtube.com/watch?v=6dem1mqneGg
16. https://stackoverflow.com/questions/52310689/use-ipython-repl-in-vs-code
17. https://www.reddit.com/r/vscode/comments/1g936fc/how_do_i_run_lines_in_the_terminal_not_the_repl/
18. https://oneuptime.com/blog/post/2026-01-24-bash-command-not-found/view
19. https://stackoverflow.com/questions/70940635/how-to-embed-spawn-an-interactive-shell-from-inside-a-python-program
20. https://stackoverflow.com/questions/17157359/running-python-script-from-bash-command-not-found-error
21. https://stackoverflow.com/questions/61982996/calling-wtforms-validations-conditionally
22. https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login
23. https://www.permit.io/blog/implement-role-based-access-control-in-flask
24. https://stackoverflow.com/questions/63549476/how-to-implement-role-based-access-control-in-flask

25. https://stackoverflow.com/questions/49615983/how-to-use-current-user-in-a-flask-wtforms-validator
26. https://wtforms.readthedocs.io/en/2.3.x/crash_course/
27. https://flask.palletsprojects.com/en/stable/patterns/wtforms/
28. https://www.reddit.com/r/flask/comments/8vhkh6/wtforms_queryselectfield_doesnt_seem_to_work_with/
29. https://stackabuse.com/flask-form-validation-with-flask-wtf/
30. https://stackoverflow.com/questions/69055545/how-to-verify-if-user-is-logged-in-before-launching-sign-in-bootstrap-modal-flas
31. https://stackoverflow.com/questions/44742438/redirecting-on-form-validation-failure-flask-wtforms
32. https://flask.palletsprojects.com/en/stable/patterns/wtforms/
33. https://stackoverflow.com/questions/63402345/cant-get-error-message-to-show-with-wtforms-formhelpers-to-show-on-flask-webpa
34. https://www.reddit.com/r/flask/comments/6dz044/ask_flask_how_to_use_redirecturl_for_when_my/
35. https://www.reddit.com/r/flask/comments/bc1ksm/af_post_form_without_redirectwtforms/
36. https://www.youtube.com/watch?v=r3Xmcdlx-Us
37. https://www.youtube.com/watch?v=P_3yc2XDssw
38. https://stackoverflow.com/questions/71734915/flask-login-authentication-fails-on-validate-in-production
39. https://stackoverflow.com/questions/67168922/flask-how-to-redirect-login-required-to-specific-route
40. https://www.reddit.com/r/flask/comments/10hmtb4/not_able_to_run_my_form_on_flask_always_keep/
41. https://stackoverflow.com/questions/25324113/email-validation-from-wtform-using-flask
42. https://wtforms.readthedocs.io/en/2.3.x/validators/
43. http://wtforms.simplecodes.com/docs/0.6.1/validators.html
44. https://exchangetuts.com/index.php/email-validation-from-wtform-using-flask-1639783691058077
45. https://debuglab.net/2024/02/28/wtforms-install-email_validator-for-email-validation-support/
46. https://wtforms.readthedocs.io/en/2.3.x/fields/
47. https://www.youtube.com/watch?v=fCAXB9Fz0vI
48. https://oneuptime.com/blog/post/2026-02-02-flask-wtf-validation/view
49. https://www.youtube.com/watch?v=QI2I9503BQ8
50. https://wtforms.readthedocs.io/en/3.0.x/validators/
51. https://flask-login.readthedocs.io
52. https://stackoverflow.com/questions/31059694/flask-how-to-prevent-to-go-to-login-page-when-already-logged-in
53. https://stackoverflow.com/questions/66760994/preventing-user-from-accessing-login-page-while-signed-in-using-flask-login/66765402
54. https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login
55. https://stackoverflow.com/questions/61939800/role-based-authorization-in-flask-login
56. https://www.reddit.com/r/learnprogramming/comments/hwvvc6/flask_login_redirect_loop/

57. https://stackoverflow.com/questions/18190067/flask-login-and-principal-current-user-is-anonymous-even-though-im-logged-in
58. http://flask-web-academy.com/article/flask-login-current_user/
59. https://www.reddit.com/r/flask/comments/199j61t/how_to_make_an_autologout_function_after_changing/
60. https://stackoverflow.com/questions/47952830/flask-login-current-user
61. https://stackoverflow.com/questions/38378102/url-redirect-in-flask-login
62. https://flask-login.readthedocs.io
63. https://stackoverflow.com/questions/25417987/how-to-redirect-users-to-different-pages-based-on-their-roles-in-flask-login/25419728
64. https://stackoverflow.com/questions/37597038/why-isnt-my-current-user-authenticated-in-flask-login
65. https://www.mitchellbusby.com/2016/08/21/tutorial-roll-your-own-user-authorisation-management-with-flask-login/
66. https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login
67. https://www.reddit.com/r/flask/comments/10knzr9/rendering_stuff_based_on_flask_user_role/
68. https://stackoverflow.com/questions/67206460/flask-login-current-user-is-authenticated-always-is-false
69. http://flask-web-academy.com/article/flask-login-current_user/
70. https://www.permit.io/blog/implement-role-based-access-control-in-flask